

Cascaded Shadow Maps — Commentaires & Sommaire de la mise à jour

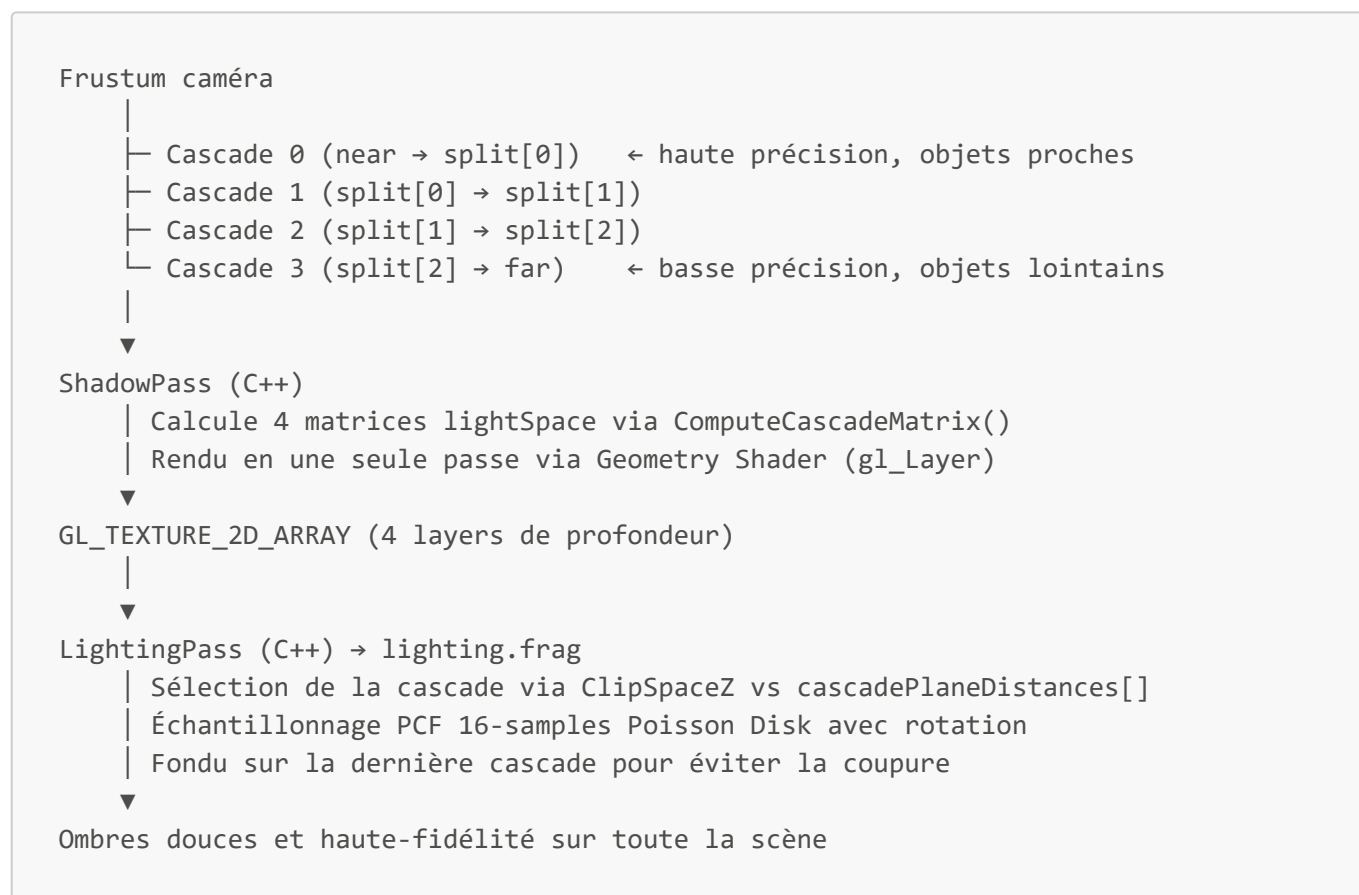
Sommaire de la mise à jour

Cascaded Shadow Maps (CSM) — Vue d'ensemble

L'ancienne implémentation utilisait **une seule shadow map 2D** avec une projection orthographique fixe centrée sur la caméra. Résultat : les ombres proches étaient pixelisées et les ombres lointaines étaient souvent hors du frustum de la shadow map.

La nouvelle implémentation découpe le frustum de la caméra en **4 sous-frustums** (cascades), chacun ayant sa propre projection orthographique ajustée au plus serré. Les cascades proches ont une haute résolution par texel, les cascades lointaines couvrent plus de surface avec moins de précision — exactement le compromis qu'il nous faut.

Architecture du pipeline CSM



Fichiers modifiés (8 fichiers)

Fichier	Changement
DirectionalLight.h	Ajout <code>NUM_CASCADES = 4</code> , renommé <code>shadowMap</code> → <code>shadowMapArray</code>

Fichier	Changement
RenderSettings.h	Remplacé <code>orthoSize/nearPlane/farPlane</code> par <code>maxShadowDistance + cascadeSplitLambda</code>
IRenderPass.h	<code>lightSpaceMatrix</code> → <code>lightSpaceMatrices[4]</code> + <code>cascadePlaneDistances[4]</code> dans <code>RenderData</code>
LightManager.cpp	<code>GL_TEXTURE_2D</code> → <code>GL_TEXTURE_2D_ARRAY</code> avec <code>glTexImage3D</code> (4 layers)
Renderer.cpp	Ajout du geometry shader <code>.geom</code> au chargement du shader <code>directional_shadow</code>
ShadowPass.h	Ajout des helpers <code>GetFrustumCornersWorldSpace</code> et <code>ComputeCascadeMatrix</code>
ShadowPass.cpp	Réécriture complète — practical split scheme, AABB par cascade, rendu layered
LightingPass.cpp	Bind <code>sampler2DArray</code> + envoi des uniforms cascade au shader

Shaders modifiés (3 fichiers)

Shader	Changement
<code>directional_shadow.vert</code>	Ne fait plus que <code>model * aPos</code> → produit du world-space pour le geometry shader
<code>directional_shadow.geom</code>	Nouveau — <code>invocations = 4</code> , applique <code>lightSpaceMatrices[gl_InvocationID]</code> , écrit <code>gl_Layer</code>
<code>lighting.frag</code>	Utilise <code>sampler2DArray</code> , sélection de cascade par <code>ClipSpaceZ</code> , PCF Poisson Disk par cascade

Concepts clés

- **Practical Split Scheme** — Interpolation logarithmique / uniforme contrôlée par `lambda` pour la répartition des cascades
- **Tight-fit AABB** — Chaque cascade calcule son propre bounding box dans l'espace lumière, pas de gaspillage de résolution
- **Texel Snapping** — Empêche le "shadow swimming" quand la caméra se déplace
- **Layered FBO** — `glFramebufferTexture` attache toutes les couches en une fois, le geometry shader route vers `gl_Layer`
- **Clip-space Z comparison** — Le fragment shader compare `ClipSpaceZ` (pré-division perspective) pour éviter un calcul view-space coûteux

Commentaires à ajouter par fichier

1. ShadowPass.h

Ligne 12 — avant la class : la doc de classe est manquante, ajouter :

```

/// @class ShadowPass
/// @brief Passe de rendu pour générer les shadow maps directionnelles en Cascaded
Shadow Mapping (CSM)
///
/// Cette passe découpe le frustum de la caméra en NUM_CASCADES sous-frustums,
/// calcule une projection orthographique ajustée pour chacun, puis rend toute la
scène
/// dans un GL_TEXTURE_2D_ARRAY via un geometry shader qui route chaque triangle
vers le layer approprié.
///
/// Utilise glCullFace(GL_FRONT) pendant le rendu pour réduire le shadow acne.

```

Ligne 45 — avant `ComputeCascadeMatrix` : le doxygen est manquant, ajouter :

```

/// @brief Construit la matrice lightView * lightProjection pour une cascade
donnée
/// @details Calcule l'AABB des coins du frustum dans l'espace lumière,
construit une
///          projection orthographique ajustée au plus serré, puis applique un
texel snapping
///          pour éviter le shadow swimming lors des mouvements de caméra.
/// @param frustumCorners Les 8 coins world-space du sous-frustum de la
cascade
/// @param lightDir La direction normalisée de la lumière (Forward du
transform)
/// @param shadowResolution La résolution de la shadow map (pour le texel
snapping)
/// @return La matrice combinée lightProjection * lightView pour cette cascade

```

2. `ShadowPass.cpp`

Ligne 31, remplacer le commentaire `// Helpers` par :

```

// =====
// Helpers – Calcul des frustums et matrices de cascade
// =====

```

Ligne 33 — avant `GetFrustumCornersWorldSpace`, ajouter :

```

/// @details Construit une matrice perspective pour le sous-frustum [nearPlane,
farPlane],
///          puis inverse la VP pour projeter les 8 coins NDC (-1..+1) en world-
space.
///          Les coins sont renvoyés dans un ordre itératif (x,y,z ∈ {0,1}).

```

Ligne 38 — au-dessus de `Matrix4x4 inv = :`

```
// Inverse de la matrice View-Projection : permet de reprojeter les coins NDC
vers le world-space
```

Ligne 54 — au-dessus du bloc `float wx = :`

```
// Multiplication manuelle inv * vec4(ndc, 1.0) avec division
perspective (w)
// Nécessaire car notre Matrix4x4::operator* ne gère que les
Vector3
```

Lignes 73-74 — remplacer le corps de la signature `ComputeCascadeMatrix` par un bloc commenté :

Après la ligne `{` (ligne 78), ajouter :

```
// --- Étape 1 : Centroïde du sous-frustum ---
// Moyenne des 8 coins pour positionner la caméra-lumière au centre du volume
```

Ligne 90 — au-dessus du bloc `Vector3 lightUp :`

```
// --- Étape 2 : Matrice de vue lumière ---
// Choix du vecteur up : si la lumière est quasi-verticale ( $|y| > 0.99$ ),
// on utilise Z comme up pour éviter la dégénérescence du cross product
```

Ligne 101 — au-dessus de `Matrix4x4 lightView = :`

```
// La caméra-lumière se place à (center - lightDir) et regarde vers center
// La distance exacte n'importe pas en projection ortho – seule la direction
compte
```

Ligne 103 — au-dessus de `float minX = :`

```
// --- Étape 3 : AABB des coins du frustum dans l'espace lumière ---
// On transforme chaque coin en light-view space et on accumule les min/max
```

Ligne 115 — au-dessus du bloc `for (const auto &c ::`

```
// Transformation manuelle des coins : lightView * corner (multiplication
colonne-major)
```

Ligne 128 — au-dessus de `float zMult = :`

```
// --- Étape 4 : Extension de la plage Z ---
// Les objets derrière le frustum de la caméra peuvent quand même projeter des
ombres
// dans le frustum. On étend donc minZ/maxZ par un facteur de sécurité.
```

Ligne 142 — au-dessus de `Matrix4x4 lightProj; :`

```
// --- Étape 5 : Construction de la projection orthographique ---
// Matrice ortho construite manuellement (column-major, convention OpenGL)
// m[0]  = 2/(r-l)      m[12] = -(r+l)/(r-l)
// m[5]  = 2/(t-b)      m[13] = -(t+b)/(t-b)
// m[10] = -2/(f-n)     m[14] = -(f+n)/(f-n)
// m[15] = 1
```

Ligne 157 — au-dessus de `Matrix4x4 cascadeVP = :`

```
// --- Étape 6 : Texel Snapping ---
// Sans cette étape, les ombres "nagent" (shadow swimming) lorsque la caméra
se déplace.
// On projette l'origine (0,0,0) dans l'espace shadow map, on l'arrondit au
texel le plus
// proche, puis on corrige le biais dans la matrice de projection.
```

Ligne 172, remplacer le commentaire `// Exécution` par :

```
// =====
// Execute – Passe principale de Cascaded Shadow Mapping
// =====
```

Ligne 174 — avant `void ShadowPass::Execute`, ajouter :

```
/// @details Étapes de la passe :
/// 1. Calcul des distances de split (practical split scheme)
/// 2. Pour chaque cascade : extraction des coins du sous-frustum et calcul de
la matrice light-VP
/// 3. Conversion des distances de split en clip-space Z (pour le fragment
shader)
/// 4. Attachement du GL_TEXTURE_2D_ARRAY au FBO (toutes les couches d'un coup)
/// 5. Envoi des 4 matrices au geometry shader qui route vers gl_Layer
```

```
/// 6. Rendu de tous les objets (pas de frustum culling par cascade pour
simplicité)
```

Ligne 190 — au-dessus du bloc `// cascade split distances` :

```
/// --- Practical Split Scheme (Parallel-Split Shadow Maps, GPU Gems 3) ---
/// Interpole entre un split logarithmique et un split uniforme.
/// log(i)      = near * (far/near)^(i/N)      - meilleure répartition
théorique
/// uniform(i) = near + (far-near) * (i/N)      - évite les cascades trop
petites proche du near
/// split(i)    = lambda * log + (1-lambda) * uniform
```

Ligne 212 — au-dessus de `// compute les 4 cascade VP matrices` :

```
/// --- Construction des 4 matrices light-space ---
/// Chaque cascade couvre [splitNear, splitFar] du frustum caméra
```

Ligne 226 — au-dessus de `// Store cascade plane distance` :

```
/// --- Conversion des distances de split en clip-space Z ---
/// Le vertex shader passe ClipSpaceZ = (proj * view * pos).z au fragment
shader.
/// Pour comparer, on doit convertir nos distances (view-space positives) en
clip-space :
/// clipZ = proj[10] * (-d) + proj[14]
/// Le signe négatif vient du fait que view-space Z est négatif vers l'avant en
OpenGL
```

Ligne 234 — au-dessus de `m_shadowFBO->Bind()` :

```
/// --- Rendu layered dans le FBO ---
/// glFramebufferTexture (sans suffixe "2D") attache TOUTES les layers du
texture array.
/// Le geometry shader écrit gl_Layer = gl_InvocationID pour router chaque
triangle
/// vers la couche correspondante. Une seule passe de draw = 4 shadow maps.
```

Ligne 248 — au-dessus de `// pass cascade matrices to geom shader` :

```
/// Envoi des matrices au geometry shader : lightSpaceMatrices[0..3]
/// Le vertex shader ne fait que model * pos (world-space),
```

```
// le geometry shader multiplie ensuite par
lightSpaceMatrices[gl_InvocationID]
```

Ligne 255 — au-dessus de `for (const auto &renderable :`

```
// Rendu de TOUS les objets de la scène (pas seulement les visibles par la
caméra)
// car un objet hors du frustum caméra peut quand même projeter une ombre
dedans
```

3. `ShadowPass.h`

Déjà traité ci-dessus.

4. `LightingPass.cpp`

Ligne 62 — au-dessus de `// uniforms CSM :`

Remplacer `// uniforms CSM` par :

```
// --- Cascaded Shadow Map uniforms ---
// On envoie les 4 matrices light-space et les 4 distances de plan de
cascade
// au fragment shader pour la sélection de cascade et la projection dans
l'espace lumière
```

Ligne 72 — au-dessus de `if (light->castsShadows && light->shadowMapArray != 0) :`

```
// Bind de la texture array CSM sur l'unité 8 (évite les conflits avec les
textures matériau)
// Le fragment shader échantillonne avec texture(shadowMapArray, vec3(uv,
layer))
```

5. `LightManager.cpp`

Ligne 107 — au-dessus de `glGenTextures :`

Remplacer (ou enrichir) le bloc de création :

```
// --- Création de la texture array pour le Cascaded Shadow Mapping ---
// GL_TEXTURE_2D_ARRAY : une texture 3D où chaque "tranche" est une shadow map
de cascade
```

```
// Chaque layer a la même résolution (shadowResolution x shadowResolution)
// Format GL_DEPTH_COMPONENT32F pour la précision en profondeur
```

Ligne 119 — au-dessus des `glTexParameteri` :

```
// NEAREST filtering : pas d'interpolation sur les valeurs de profondeur (le
PCF se fait dans le shader)
// CLAMP_TO_BORDER avec couleur blanche (1.0) : hors-frustum → profondeur max
→ pas d'ombre
```

6. `IRenderPass.h`

Le fichier est déjà bien commenté. Rien à ajouter.

7. `DirectionalLight.h`

Ligne 28 — au-dessus de `unsigned int shadowMapArray`, enrichir :

```
/// @brief Identifiant OpenGL de la shadow map array (GL_TEXTURE_2D_ARRAY,
NUM_CASCADES layers)
/// Initialisé à 0, créé par LightManager::CreateShadowResourcesFor()
```

8. `RenderSettings.h`

Le fichier est déjà bien commenté. Rien à ajouter.

9. `Renderer.cpp`

Ligne 49 — au-dessus du `GetShader("directional_shadow", ...)` :

```
// Chargement du shader d'ombres avec le geometry shader pour le rendu CSM
layered
// Le geometry shader (directional_shadow.geom) route chaque triangle vers
gl_Layer = cascade index
```

Résumé

Fichier	Nb de commentaires à ajouter
<code>ShadowPass.cpp</code>	~15 blocs (fichier le plus dense)

Fichier	Nb de commentaires à ajouter
ShadowPass.h	2 blocs (doc de classe + <code>ComputeCascadeMatrix</code>)
LightingPass.cpp	2 blocs
LightManager.cpp	2 blocs
DirectionalLight.h	1 bloc
Renderer.cpp	1 bloc
IRenderPass.h	rien
RenderSettings.h	rien

Bravo pour la mise à jour, les CSM c'est un vrai milestone pour un moteur de rendu !